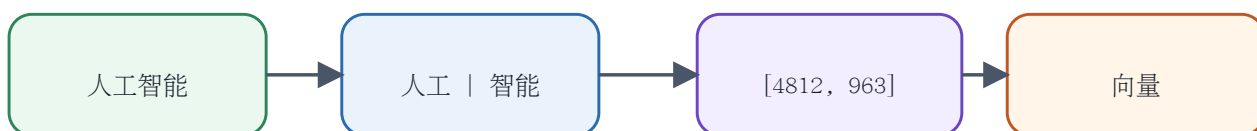


大语言模型零基础课

第2讲：Token

大语言模型眼里的文字到底是什么？

从“人看到文字”走到“模型看到 Token ID”



适合对象：零基础。默认不要求 NLP、Transformer、线性代数基础。

本讲原则：先理解 Token 为什么存在，再理解 tokenizer 如何切分，最后连接上下文、成本与 Agent Systems。

本讲材料版本：2026 · PDF 原生排版

本讲学习目标

核心概念 学完后你应该真正会解释

Token 不是“一个字/一个单词”的别名，而是 tokenizer

词表中的离散符号。你应该能解释：文字为什么要先变 Token; Token、Token ID、Embedding 的区别; BPE 的基本直觉; 为什么同一句话在不同模型里 Token 数会不同; Token 数为什么影响上下文、费用、延迟和 Agent 系统。

目录

- 01 先纠正第一直觉：模型看不到“文字”
- 02 Token 到底是什么？
- 03 为什么不用“一个字”或“一个单词”就完了？
- 04 Tokenizer: 文字是怎样被切成 Token 的？
- 05 BPE: 最值得掌握的分词直觉
- 06 Token ID 与 Embedding: 离散符号如何进入神经网络
- 07 中文、英文、数字、代码、emoji 为什么切法不同？
- 08 Special Token 与 Chat Template
- 09 Token、上下文窗口、API 成本与速度
- 10 Token 与 Agent Systems 的科研关系
- 11 常见误区与边界
- 12 动手实验
- 13 术语表
- 14 练习与答案
- 15 一页总结与下一讲预告

1

模型看不到“文字”

计算机最终处理的是数字，不是你眼中的汉字和单词

你看到“人工智能”四个汉字，会直接产生语义。神经网络却不能把“人工智能”这几个字符原样塞进矩阵乘法。现代 LLM 的第一道接口，是把文本转换成一串离散编号。

模型处理文字时的四层接口（示意，不代表真实切分）



核心概念 先记住这一条主线

Text → Token → Token ID → Embedding → Transformer。Tokenization 负责“切块并编号”；Embedding

再把编号变成神经网络可计算的向量。后面所有注意力计算，处理的已经不是原始文字。

1.1 为什么不能直接把 Unicode 编码当语义？

计算机当然能用 Unicode/UTF-8 表示文字，但“能存储这个字符”与“知道它是什么意思”是两回事。例如字符“猫”可以对应一个编码值，这个编码值本身只负责身份标识，不意味着数字大小包含“猫科动物”的语义。LLM 需要一个可学习的表示层，所以通常先得到 Token ID，再查 Embedding 表获得向量。

补充说明 底层区分

字符编码解决“这个字符怎样存进电脑”；tokenizer解决“怎样把文本切成模型词表里的离散单元”；embedding解决“怎样把离散 ID 映射成可学习的连续向量”。三件事不要混。

2

Token 到底是什么？

它是“模型词表中的一个离散符号”

最稳妥的定义是：Token 是 tokenizer

根据自己的词表与编码规则，把文本表示成的一串离散单位。一个 token 可能对应一个完整单词、一个词的一部分、一个汉字、几个汉字、空格与单词的组合、标点、代码片段，甚至某些字节序列。

常见误区 Token $\neq$ 单词；Token $\neq$ 汉字

例如教学用 tokenizer 可能把 “unbelievable” 切成 “un | believe | able”，也可能整词作为一个 token。中文 “人工智能” 可能被切成 “人工 | 智能”，也可能是其他方式。真实切分由具体模型的 tokenizer 决定。

同一段文字，可以选择不同“切块粒度”

按字符

u | n | b | e | l | i | e | v | a | b | l | e

按单词

unbelievable

子词 Token

un | believe | able

现代 LLM 常用子词/字节级方法，在“词表大小、泛化、压缩效率”之间折中。

2.1 Token 是模型与语言之间的“接口标准”

如果模型词表中共有 V 个 token，我们可以把每个 token 编成 0 到 $V-1$ 的整数。神经网络不需要理解“这个 ID 看起来像什么”，只需要知道这个 ID 对应哪一行 embedding。于是文本处理问题被转换为离散序列问题。

示意：

“AI is useful.” $\rightarrow$ [1024, 381, 7259, 13]

这里的数字完全是教学示意，不代表任何真实模型。关键是：同一个 token 在词表里有稳定 ID。

3

为什么不直接按“字”或“词”？

Tokenizer 本质上是在做压缩效率与泛化能力的折中

方案 A: 一个字符一个 Token

优点: 不会遇到“生词”问题, 词表可以较小; 缺点: 常见词也会被拆得很碎, 序列变长。序列越长, Transformer 要处理的时间步越多。

方案 B: 一个完整单词一个 Token

优点: 常见句子很短; 缺点: 词表会极大, 而且新词、拼写变化、复合词、代码标识符很难处理。比如 run、runs、running、runner 是否都单独占一个词表项?

方案 C: 子词 / 字节级 Token

现代 LLM 通常采用某种子词或字节友好的方法。高频片段可以合并成较长 token, 罕见词则拆成较小片段。这样既不会让词表无限膨胀, 也能覆盖未见过的新字符串。

例子 一个压缩比喻

把 tokenizer 想成“文本压缩字典”。高频短语值得放进字典里, 用一个编号表示; 极少见的字符串不值得专门占一个编号, 就拆成更小单元组合出来。词表是在有限容量下对常见模式做压缩。

科研连接 为什么对科研很重要

同样 10,000 个可见字符, 如果某 tokenizer 编成 6,000 tokens, 而另一个编成 12,000 tokens, 那么它们消耗的上下文预算、推理计算和 API 费用都可能明显不同。Tokenization 不是无关紧要的前处理。



Tokenizer 怎样工作？

训练 tokenizer 与使用 tokenizer 是两个阶段

4.1 阶段一：先“学”一个词表

模型训练之前，通常会先用大量文本训练 tokenizer。它要回答：哪些字符串片段值得拥有独立 token？目标通常是在固定词表规模下，让训练语料能被更高效地编码，同时保留对任意输入的覆盖能力。

4.2 阶段二：把新文本编码成 Token ID

tokenizer

确定以后，用户输入到来时并不会重新学习词表，而是使用固定词表和固定规则，把新文本编码成 token IDs。解码时则做反方向：Token IDs → 文本片段 → 可读字符串。

方向	输入	输出	典型名称
编码	文本	Token IDs	encode / tokenize
解码	Token IDs	文本	decode

核心概念 注意：tokenizer 通常不是神经网络“大脑”

tokenizer 一般是模型前后的确定性编码组件。给定同一版本 tokenizer 和同一字符串，通常会得到相同 token IDs。真正的大规模神经网络从这些 IDs 的 embedding 开始处理。

5

BPE：掌握一个代表性原理

Byte Pair Encoding 的核心是“频繁相邻片段合并”

你不需要马上掌握所有 tokenizer 算法。先把 BPE (Byte Pair Encoding) 的直觉弄懂，就已经抓住了“子词词表如何形成”的核心思想。

BPE 的直觉：把“经常一起出现”的相邻片段逐步合并

初始: 1 | o | w | e | r

统计: 发现 1+o、1o+w 等相邻组合反复出现

合并 1: 1o | w | e | r

合并 2: 1ow | e | r

词表结果: 常见片段可能成为一个 token: 1ow、er、ing...

这是教学化简图。真实 tokenizer 的训练、字节处理、正则预切分和编码策略会因模型而异。

5.1 一个极简例子

假设训练语料里反复出现 lower、lowest、low。开始时可以把文本拆得很细。统计后会发现 1-o、1o-w 组合高频，于是不断合并，最终 “1ow” 很可能成为一个常见 token。与此同时，“lowest” 仍可表示成 “1ow + est” 或其他片段。

例子 BPE 为什么有效？

因为自然语言存在大量重复的词根、前后缀、常用词和字符组合。把高频组合合并，相当于用有限词表对文本做更高效的可逆编码。

5.2 真实世界没有一种唯一 tokenizer

不同模型可能使用 BPE、byte-level BPE、SentencePiece、Unigram 或其他变体，还会有不同的正则预切分、字节回退、特殊 token 设计。因此你不能拿模型 A 的 token 数，直接断言模型 B 也一样。

科研连接 科研写作注意

论文如果报告 token budget、context length、prompt length 或成本，最好明确使用了哪个模型/哪个 tokenizer。否则“1000 tokens”的实际文本容量可能跨模型并不等价。

6

Token ID 与 Embedding

ID 只是索引, Embedding 才进入连续数学空间

假设 tokenizer 把文本切成三个 token, 并得到 IDs:

[381, 7259, 13]

这三个整数不是“语义分数”。381 并不比 13

更“高级”。它们更像图书馆索书号: 只负责定位。模型拿到 ID 后, 会到一个可训练的 embedding matrix 中取对应的一行向量。

示意:

ID 381 $\rightarrow$ [0.12, -0.44, 0.03, ...]

这个向量才会进入后续的 Transformer 计算。训练过程中, embedding 矩阵本身也会更新, 于是不同 token 的表示逐渐具有有用的几何和语义结构。

常见误区 不要把 Token ID 当“词义编码”

Token ID 本身通常只是离散编号。真正承载可学习表示的是 embedding 及后续网络状态。把“ID=381”解释成某种语义数值是错误的。

6.1 为什么下一讲才重点讲 Embedding?

因为 Embedding

会引入“向量、维度、相似度、可学习参数”等新概念。本讲只需要建立接口关系: tokenizer 把字符串映射到整数; embedding 把整数映射到向量。

7

不同文本为什么 Token 数差很多？

频率、语言、空格、标点、数字、代码都会影响切分

文本类型	为什么会这样	你要记住
常见英文短语	高频模式可能被较长 token 表示	通常较省 token，但取决于词表
罕见英文词/人名	可能拆成多个子词	序列变长
中文	可能按字、词片段或字节组合	不同 tokenizer 差异较大
数字	长数字可能被拆成多段	不要假设 “一个数字=一个 token”
代码	缩进、符号、标识符都参与编码	代码模型 tokenizer 往往会特别优化
emoji/特殊符号	可能对应一个或多个 token/字节片段	视觉上的 “一个符号” 不等于一个 token

常见误区 最容易犯的错误

不要背 “1 token $\approx$ 4 个英文字符” “1 token $\approx$ 0.75 个单词” 之类经验公式并把它当定律。它们最多是特定模型/语料上的粗略估计。需要精确 token 数时，必须用目标模型的 tokenizer 实际编码。

7.1 空格也可能参与 Token

很多 tokenizer 会把 “前导空格 + 单词” 视为一个常见片段，因此 “hello” 和 “ hello” 可能不是同一个 token。对代码尤其重要，因为空格、换行和缩进本身就是结构。

7.2 Token 边界不等于语义边界

一个词可能被拆成几块，一个 token

也可能包含空格和多个字符。模型后续通过上下文注意力把这些片段重新组合成复杂表示，所以 tokenizer 切分得 “不像人” 并不意味着模型必然无法理解语义。

8

Special Token 与 Chat Template

系统角色、消息边界、结束标记也需要编码

聊天界面里的“角色”，最终也要被编码成 token 序列



实际模型会用自己的 chat template / special tokens 编码角色边界；不同模型格式不同。

聊天产品看起来有 System / User / Assistant 三种消息，但底层模型最终需要的是一条 token 序列。为了让模型知道“谁说了什么、哪里开始、哪里结束”，不同模型会定义特殊 token 或特定文本模板。

BOS: Beginning of Sequence, 序列开始标记（是否使用取决于模型）。

EOS: End of Sequence, 序列结束标记。

PAD: Padding, 用于批处理时补齐长度（并非所有推理场景都需要）。

角色/消息边界 token: 用于标记 system、user、assistant
或消息结束；具体命名完全由模型决定。

科研连接 为什么系统提示词也占上下文？

因为 system prompt、user message、assistant history、工具描述、工具结果，最终都要被编码成 token。对 Agent 来说，工具 schema、记忆和历史轨迹可能占掉大量上下文预算。

常见误区 不要手写假想 special token

不同模型的 chat template 不同。实际调用 API 或本地模型时，应使用官方/对应 tokenizer 提供的 chat template，而不是凭感觉拼 “...”。

9

Token 与上下文、费用、速度

Token 是 LLM 系统里的“计算货币”之一

Token 数量为什么重要？

上下文窗口

能塞进多少信息

API 计费

输入/输出通常按 token

推理延迟

更多 token = 更多计算

Agent 成本

多轮+工具结果快速累积

9.1 Context Window 的单位为什么是 Token？

因为 Transformer 接收的是 token 序列，而不是“页数”或“汉字数”。所谓 128K、1M context，通常是在说明模型一次能处理的 token 数上限或设计目标。它不是 128K 个中文字符，也不是 1M 个单词。

9.2 API 为什么按 Token 计费？

模型计算量与输入、输出 token 数高度相关。商业 API 因此通常分别统计 input tokens 与 output tokens，并按相应价格计费。具体价格会变化，本讲不记数字，只记机制。

9.3 长上下文为什么贵？

更多输入 token 意味着要做更多 embedding、attention、前馈网络等计算，还会增加 KV Cache 等推理内存需求。不同架构和优化会改变复杂度，但“token 越多，系统成本通常越高”这个直觉很重要。

科研连接 Agent 成本为什么容易爆炸

单轮聊天可能只有几千 tokens；Agent 循环却会不断加入计划、历史、工具输出、网页、代码、日志和多 Agent 消息。若每一轮都把全部历史重放，token 使用量会随轨迹快速增长。



Token 与 Agent Systems

对你后面科研最值得保留的系统视角

10.1 Context Budget = 有限系统资源

Agent 不是无限记忆。所有放进 prompt 的信息都竞争有限 context budget。工具文档太长、历史太多、memory 检索太宽，都会挤压真正需要推理的信息。

10.2 通信协议的成本可以用 Token 测量

多 Agent 系统里，Agent 之间每发一大段自然语言都要占 token。于是“沟通更多是否一定更好”变成一个可实验问题：额外通信带来的准确率收益，是否值得付出的 token、延迟和错误传播成本？

10.3 Tokenization 还能制造系统偏差

如果不同语言、代码风格或数据格式在某 tokenizer 下编码效率不同，那么“相同字符数”并不代表“相同计算预算”。跨语言 benchmark、成本比较和公平资源分配时必须留意这一点。

科研连接 一个可以直接带进论文阅读的指标

以后看到 Agent 论文，不只看 accuracy / success rate。还要留意：input tokens、output tokens、total tokens、number of turns、tool calls、latency、cost。系统科学关心的是“性能-资源-可靠性”的联合关系。



常见误区

这一讲最容易混淆的 9 件事

常见误区 误区1: 一个汉字就是一个 Token

不一定。取决于 tokenizer 词表和编码规则。

常见误区 误区2: 一个英文单词就是一个 Token

不一定。常见词可能一个 token, 罕见词可能拆成多个子词。

常见误区 误区3: Token ID 的数值大小代表语义距离

错误。ID 主要是索引; 语义结构来自 embedding 与网络状态。

常见误区 误区4: Tokenizer 会根据当前句子临时学习新词

通常不会。模型部署时词表和规则是固定的。

常见误区 误区5: 不同模型的 Token 数可以直接比较

要谨慎。tokenizer 不同, 编码粒度可能不同。

常见误区 误区6: 视觉上一个 emoji 就等于一个 Token

不一定, 可能涉及多个 Unicode code points 或多个字节片段。

常见误区 误区7: 上下文 1M = 100 万字

错误。单位是 token。

常见误区 误区8: 压缩 prompt 只会省钱, 不影响性能

不一定。压缩可能删掉关键证据, 也可能减少噪声; 这是系统设计权衡。

常见误区 误区9: Tokenization 只是工程细节, 不影响科研
错误。它会影响预算、长度、成本、跨语言公平性和实验可复现性。

12

动手实验

不用任何在线服务，也能把核心机制跑一遍

动手实验 实验 1: 手工设计一个玩具词表

词表: { “人工” , “智能” , “人工智能” , “很” , “强” , “。” }。尝试编码 “人工智能很强。”。如果采用 “优先匹配最长 token” 的玩具规则, 结果可以是 [人工智能] [很] [强] [。]。然后删掉 “人工智能” 这个词表项, 再编码一次, token 数会增加。

实验 2: 比较 “字符数” 和 “Token 数” 为什么不是一回事

试着对下面四类字符串做人工切分, 并问自己: “如果我是 tokenizer 设计者, 哪些高频片段值得合并?”

自然语言: machine learning is useful

中文: 大语言模型正在生成文本

数字: 20260928123456

代码: for i in range(10):

实验 3: 运行随材料附带的 toy_tokenizer_demo.py

材料包里附了一个纯 Python 的 “最长匹配玩具 tokenizer”。它不是任何真实模型 tokenizer, 目的只是让你直观看到: 词表改变 → 切分改变 → Token ID 序列改变。不需要安装第三方库。

动手实验 实验时最重要的观察

不要关注玩具 ID 的具体数字, 而要观察: 同一字符串的字符没有变, 但由于词表不同, token 个数和边界会变。真实 LLM 的 tokenizer 本质上也是先固定 “离散符号系统”, 再把文本映射进去。



术语表

读论文和 API 文档时会反复出现

术语	解释
Token	tokenizer 输出的离散符号单位。
Tokenizer	负责文本与 token/token IDs 之间编码和解码的组件。
Vocabulary / Vocab	词表; 模型 tokenizer 可使用的全部 token 集合。
Vocabulary Size	词表大小, 即 token 类型总数。
Token ID	某 token 在词表中的整数编号, 主要充当索引。
Subword	子词单元; 介于字符与完整单词之间的片段。
BPE	Byte Pair Encoding; 通过高频相邻片段合并构造词表的经典思想。
SentencePiece	一种将分词问题处理为可直接从原始文本训练子词模型的工具/方法体系。
Special Token	承担序列边界、角色、填充等控制功能的特殊 token。
BOS / EOS	序列开始 / 结束 token (具体模型可能使用或不使用)。
Embedding	把 token ID 映射到连续向量的可学习表示。
Context Window	一次推理可处理的 token 上下文容量。
Input Tokens	请求中送入模型的 token 数。
Output Tokens	模型生成的 token 数。
Chat Template	把 system/user/assistant 等结构化消息序列化为模型可接受 token 序列的模板。

14

练习与答案

重点检验你是否真正理解接口层次

练习题

1. 为什么说 Token 不是“单词”的同义词？
2. Text $\rightarrow$ Token $\rightarrow$ Token ID $\rightarrow$ Embedding 四步分别解决什么问题？
3. Token ID=9000 是否比 Token ID=1000 表示“更重要/更高级”？
4. 为什么不用“所有完整单词都作为 token”的方案？
5. 用自己的话解释 BPE 的核心直觉。
6. 为什么同一句中文在不同模型里 token 数可能不同？
7. 为什么 1M context 不等于 100 万汉字？
8. System prompt、工具描述为什么也会消耗上下文？
9. 多 Agent 自由聊天为什么可能产生很高 token 成本？
10. 论文比较两个模型“每题固定 2000 tokens”是否一定代表给了相同信息容量？

参考答案

1. 因为 token 的边界由 tokenizer 词表决定，可以是词、子词、字符、空格+词、符号或字节片段。
2. Text 是人类字符串；Token 是词表中的离散符号；Token ID 是该符号的整数索引；Embedding 是神经网络使用的连续向量。
3. 不能。ID 通常只是索引编号，大小本身没有这种语义。
4. 词表会爆炸，而且无法优雅处理新词、词形变化、代码标识符和罕见字符串。

5. 从小片段开始，反复统计并合并高频相邻片段，让常见模式用更少 token 表示。
6. 因为模型可能使用不同词表、算法、训练语料和预切分规则。
7. context 单位是 tokenizer 编码后的 token，而字符到 token 没有固定 1:1 映射。
8. 它们最终都要被序列化为模型输入 token，与用户文本竞争同一个 context budget。
9. 每轮消息、计划、工具结果和其他 Agent 输出都会追加或重放，通信越长、轮次越多，输入输出 token 会迅速累积。
10. 不一定。不同 tokenizer 的 2000 tokens 可能覆盖不同数量的字符/信息；公平比较还需明确 tokenizer 和实际文本。



一页总结与下一讲

把 Token 放进完整 LLM 数据通路

本讲必须留下的 10 个结论

核心概念
1. 神经网络不能直接拿“人眼看到的文字”做矩阵计算；文本先要离散化。
核心概念
2. Token 是 tokenizer 词表中的离散单位，不等于字，也不等于单词。
核心概念
3. Tokenizer 决定文本怎样切分；不同模型可以完全不同。
核心概念
4. 子词方法是在词表大小、序列长度和未知字符串覆盖之间做折中。
核心概念
5. BPE 的核心直觉是把经常相邻出现的片段逐步合并。
核心概念
6. Token ID 是索引，不是语义大小；Embedding 才把 ID 映射成可学习向量。
核心概念
7. 中文、英文、数字、代码、emoji 的 token 数都不能仅凭视觉字符数判断。
核心概念

8. System/User/Assistant、工具描述和历史最终也都变成 token。

核心概念

9. Context、API 成本、延迟和 Agent 通信开销都与 token 强相关。

核心概念

10. 需要精确统计时，必须使用目标模型对应的 tokenizer。

下一讲: Embedding——文字怎样变成“有意义的向量”？

下一讲会开始引入最少量的向量直觉：什么叫维度？为什么“猫”和“狗”可能在向量空间里更接近？Embedding 是怎样训练出来的？Token Embedding 与上下文文化表示有什么区别？这将是正式进入神经网络内部的第一步。

例子 预习问题

如果 Token ID 只是任意整数编号，那么模型凭什么知道“猫”和“狗”语义接近？答案就在 Embedding。